

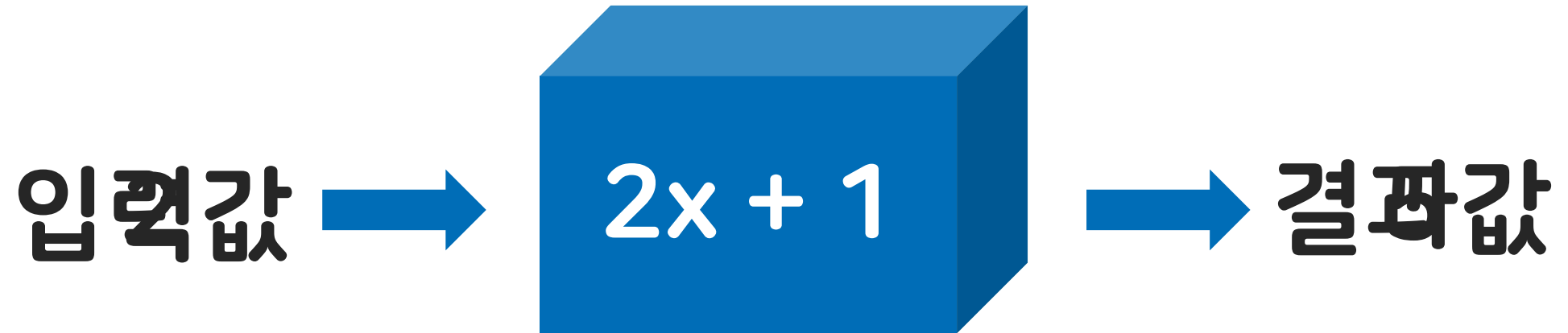


이도연 연구원



학습목표

1. 메소드의 개념과 필요성을 설명할 수 있다.
2. 메소드 구조를 만들고 활용할 수 있다.
3. 메소드 오버로딩을 이해한다.





어떤 작업을 수행하기 위한 **명령문의 집합** 여러 줄의 코드를 하나로 묶어서 표현한 형태

```
System.out.print("아이디 입력 : ");  
String id = sc.next();  
System.out.print("비밀번호 입력 : ");  
String pw = sc.next();  
if(id.equals("smhrd")&&pw.equals("12345")) {  
    System.out.println("로그인성공!");  
}else {  
    System.out.println("로그인실패!");  
}
```



반복되는 코드 **최소화!** 유지보수 편함

접근제한자

리턴 타입

메소드 이름

매개변수

```
public int addNumber(int num1, int num2) {  
    int result = num1+num2;  
    return result;  
}
```

반환데이터

접근제한자

```
[public] int addNumber(int num1, int num2) {  
    int result = num1+num2;  
    return result;  
}
```

메소드의 접근 범위를 설정해주는 키워드

public(어느 클래스에서나 접근가능), private(현재 클래스에서만 접근 가능)

리턴타입

```
public [int] addNumber(int num1, int num2) {  
    int result = num1+num2;  
    return result;  
}
```

메소드의 수행결과를 **어떤 타입(자료형)으로 반환할 것인지** 알려주는 것

리턴 타입이 있다면 반드시 내부에 **return**문을 사용하여 **결과값 반환**

아무 결과 값도 반환하지 않을 경우에는 **void 키워드** 사용(return 사용 안함)

매개변수

```
public int addNumber(int num1, int num2) {  
    int result = num1+num2;  
    return result;  
}
```

매개변수의 개수에는 제한이 없음(매개변수가 없어도 됨)

매개변수는 반드시 자료형 명시

Java™ 메소드 사용하기

```
public static void main(String[] args) {
```

```
    int value = addNumber(8, 12);  
    System.out.println("연산결과 : " + value);
```

```
}
```

```
public static int addNumber(int num1, int num2) {  
    num1 = 8, num2 = 12
```

```
    int result = num1 + num2;  
    return result;
```

```
}
```

return 키워드를 만나면 결과값 반환

메소드 명을 통해 사용 가능

num1 = 8, num2 = 12

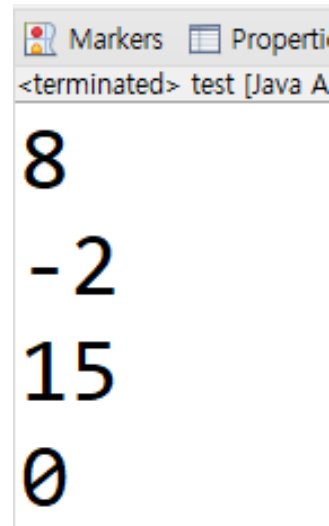
1. 메소드 예제 - 사칙연산

1. 덧셈, 뺄셈, 나눗셈, 곱셈이 가능한 메소드 4개를 생성하세요.
2. 아래와 같이 메소드를 사용하고 결과값을 출력하세요.

```
public static void main  
{  
    add(3, 5);  
    sub(3, 5);  
    mul(3, 5);  
    div(3, 5);  
}
```

Tip!

메소드 사용하는 코드를 통해
메소드 명, 리턴 타입,
매개변수 유추하기!



Markers Property
<terminated> test [Java A
8
-2
15
0

Java 메소드 빠르게 생성하기

```
public static void main
```

```
    add(3,5);
```

```
    sub(3,5);
```

```
    mul(3,5);
```

```
    div(3,5);
```

```
}
```

 The method div(int, int) is undefined for the type test

1 quick fix available:

 Create method 'div(int, int)'

Press 'F2' for focus

해석!

정수형 매개변수 2개를 받는
div라는 메소드가 존재하지 않아!

```
private static void div(int i, int j) {  
    // TODO Auto-generated method stub  
}
```

클릭 시 메소드 틀 생성!

해석!

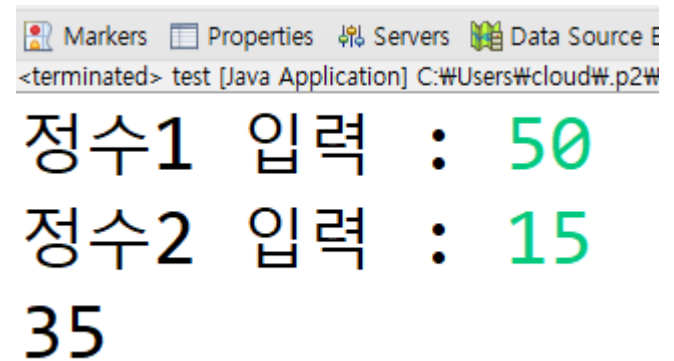
메소드 생성하겠다!

2. 메소드 예제 - 계산기

1. 정수형 num1과 num2를 입력 받고, 문자형 op를 선언해 원하는 연산자를 넣으세요.
2. num1과 num2를 op에 맞게 연산하여 최종 값을 반환해주는 cal메소드를 만드세요.

단, 빼기를 수행할 때는 더 큰 수에서 작은 수를 빼세요!

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    System.out.print("정수1 입력 : ");  
    int num1 = sc.nextInt();  
    System.out.print("정수2 입력 : ");  
    int num2 = sc.nextInt();  
    char op = '-';  
    System.out.println(cal(num1, num2, op));  
}
```



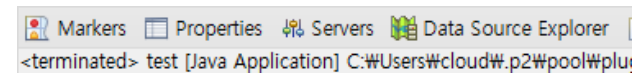
```
<terminated> test [Java Application] C:\Users\cloud\p2\W  
정수1 입력 : 50  
정수2 입력 : 15  
35
```

3. 메소드 예제 - 10에 더 가까운 수 구하기

1. 정수형 num1과 num2를 입력 받으세요.
2. num1과 num2중 10에 더 가까운 수를 반환하는 close10메소드를 생성하세요.

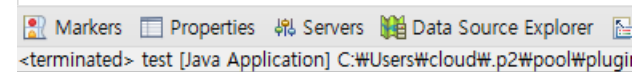
단, 두 숫자 모두 10과의 차이가 같다면 0을 반환하세요!

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    System.out.print("정수 입력 : ");  
    int num1 = sc.nextInt();  
    System.out.print("정수 입력 : ");  
    int num2 = sc.nextInt();  
    int result = close10(num1, num2);  
    System.out.println("10에 가까운 수 : " + result);  
}
```



Markers Properties Servers Data Source Explorer
<terminated> test [Java Application] C:\Users\cloud#.p2\pool\plugi

정수 입력 : 15
정수 입력 : 5
10에 가까운 수 : 0



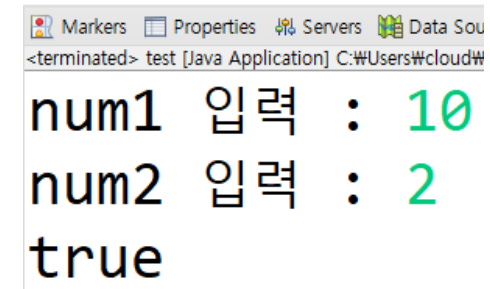
Markers Properties Servers Data Source Explorer
<terminated> test [Java Application] C:\Users\cloud#.p2\pool\plugi

정수 입력 : -1
정수 입력 : -5
10에 가까운 수 : -1

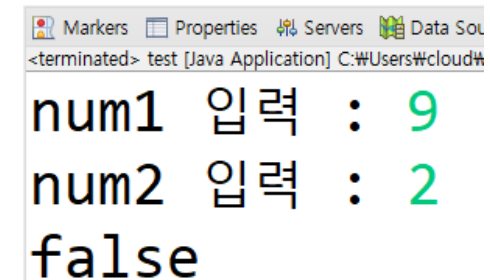
4. 메소드 예제 - 완전 수 구하기(level up!)

1. num2가 num1의 약수인지 확인하여 약수라면 true, 아니라면 false를 반환하는 isDivisor 메소드를 만들어주세요.

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    System.out.print("num1 입력 : ");  
    int num1 = sc.nextInt();  
    System.out.print("num2 입력 : ");  
    int num2 = sc.nextInt();  
    boolean divisor = isDivisor(num1, num2);  
    System.out.println(divisor);  
}
```



```
Markers Properties Servers Data Sou  
<terminated> test [Java Application] C:\Users\cloud#\br/>num1 입력 : 10  
num2 입력 : 2  
true
```

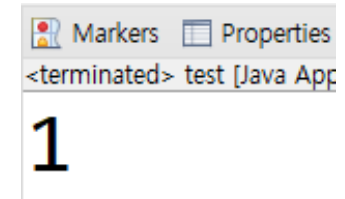


```
Markers Properties Servers Data Sou  
<terminated> test [Java Application] C:\Users\cloud#\br/>num1 입력 : 9  
num2 입력 : 2  
false
```

4. 메소드 예제 - 완전 수 구하기(level up!)

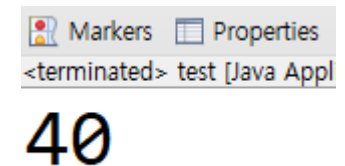
2. 자신을 제외한 약수의 총합을 구하는 getSum 메소드를 작성하세요.

```
public static void main(String[] args) {  
    System.out.println(getSum(7));  
}
```



Markers Properties
<terminated> test [Java App
1

```
public static void main(String[] args) {  
    System.out.println(getSum(44));  
}
```

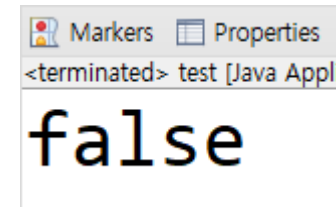


Markers Properties
<terminated> test [Java Appl
40

4. 메소드 예제 - 완전 수 구하기(level up!)

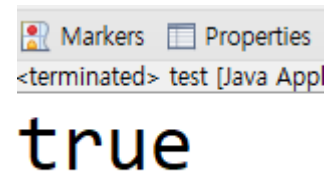
3. 입력받은 매개변수가 완전수라면 true, 아니면 false를 반환하는 isPerfect 메소드를 생성하세요.

```
public static void main(String[] args) {  
    System.out.println(isPerfect(7));  
}
```



Markers Properties
<terminated> test [Java Appl
false

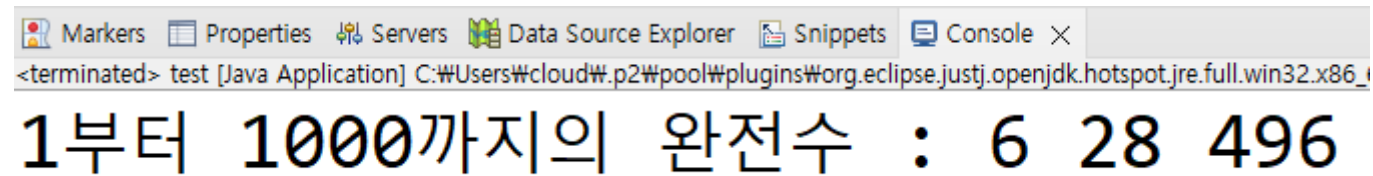
```
public static void main(String[] args) {  
    System.out.println(isPerfect(6));  
}
```



Markers Properties
<terminated> test [Java Appl
true

4. 메소드 예제 - 완전 수 구하기(level up!)

4. 1부터 1000까지의 숫자 중 완전수를 모두 출력하세요.



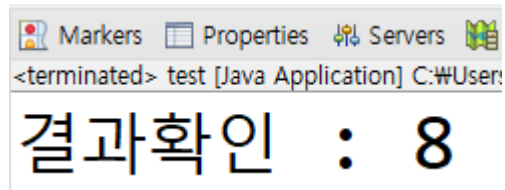
```
Markers Properties Servers Data Source Explorer Snippets Console ×  
<terminated> test [Java Application] C:\Users\cloud\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_1  
1부터 1000까지의 완전수 : 6 28 496
```

**메소드를 사용하여 코드를 생성함으로써
이미 만들어진 코드 재사용 가능!**

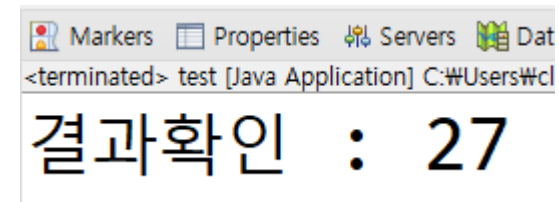
숙제. 메소드 예제 - 제공 수 구하기

1. 정수형 base와 n을 입력 받으세요.
2. base의 n제곱 만큼 값을 반환하는 powerN 메소드를 작성하세요

```
public static void main(String[] args) {  
    int base = 2;  
    int n = 3;  
    int result = powerN(base,n);  
    System.out.println("결과확인 : "+result);  
}
```



```
public static void main(String[] args) {  
    int base = 3;  
    int n = 3;  
    int result = powerN(base,n);  
    System.out.println("결과확인 : "+result);  
}
```



bonus. 메소드 예제 - 피보나치

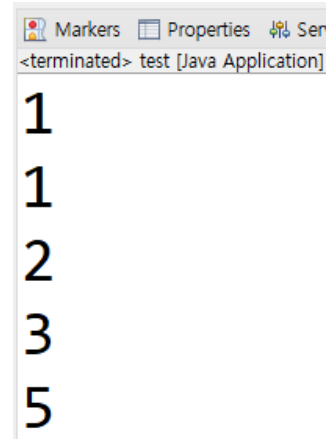
1. 피보나치 수열의 n번째 항을 출력하는 pibo 함수를 생성하세요.

피보나치 수열이란?

: 첫째, 둘째 항은 1이며 그 뒤의 모든 항은 바로 앞 두항의 합인 수열

ex) 1,1,2,3,5,8

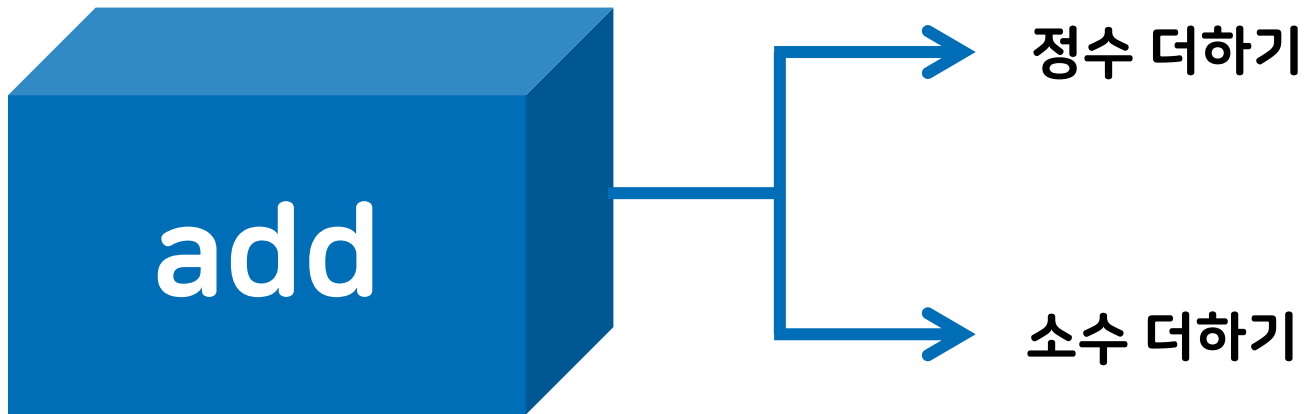
```
public static void main(String[] args) {  
    System.out.println(pibo(1));  
    System.out.println(pibo(2));  
    System.out.println(pibo(3));  
    System.out.println(pibo(4));  
    System.out.println(pibo(5));  
}
```



Markers Properties Ser
<terminated> test [Java Application]
1
1
2
3
5

메소드 오버로딩이란?

메소드의 이름은 같지만 매개변수를 다르게 함으로써
서로 다른 메소드를 만드는 기법

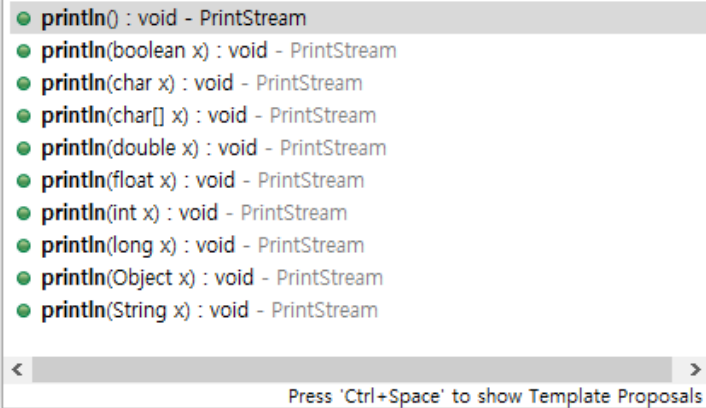


매개변수의 개수 및 매개변수의 데이터 타입으로 구분함

메소드 오버로딩의 조건

1. 메소드 이름이 같아야 한다.
2. 매개변수의 개수 혹은 타입(자료형)이 달라야 한다.
3. 매개변수와 메소드 이름이 동일하고 리턴 타입만 다른 경우는 오버로딩이 아니다.

```
System.out.println
```



메소드 이름의 낭비 방지!

개발자가 편해짐!



다음시간에 배울 내용

객체지향프로그래밍(OOP)
